Aitor Ruiz Bautista

Sistema mulitrobot : ROS2

A lo largo de esta práctica se va a desarrollar un sistema multirobot mediante el sistema ROS2. Primero se hará la implementación de un único robot en un mundo gazebo y con un visor en RViZ.

Ejecución de la simulación

Para recrear las dos simulaciones hay que seguir los siguientes pasos:

Simulación con un solo robot

Ejecutar los source correspondientes:

- $source /opt/ros/jazzy/setup.bash

Dentro de la carpeta descomprimida /entrega_aitor:

- colcon build (para recompilar)

- $source install/setup.bash

Lanzar entorno:

- $ros2 launch robot_bringup robot_bringup.launch.py

Simulación con múltiples robots

Ejecutar igual que antes pero lanzar el siguiente entorno:

- $ros2 launch multirobot_bringup multirobot.launch.py num_robots:=4

(El parámetro num_robots si no se pone viene por defecto en 2)

Desarrollo de la práctica

Para comenzar, se requiere la creación de un workspace y un paquete de ROS2.

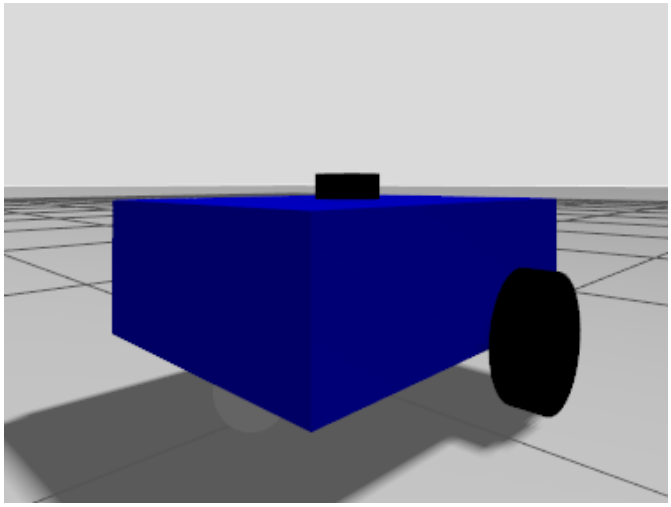
$mkdir -p ~/multirobots_ros/src

$ros2 pkg create --build-type ament_python practica1

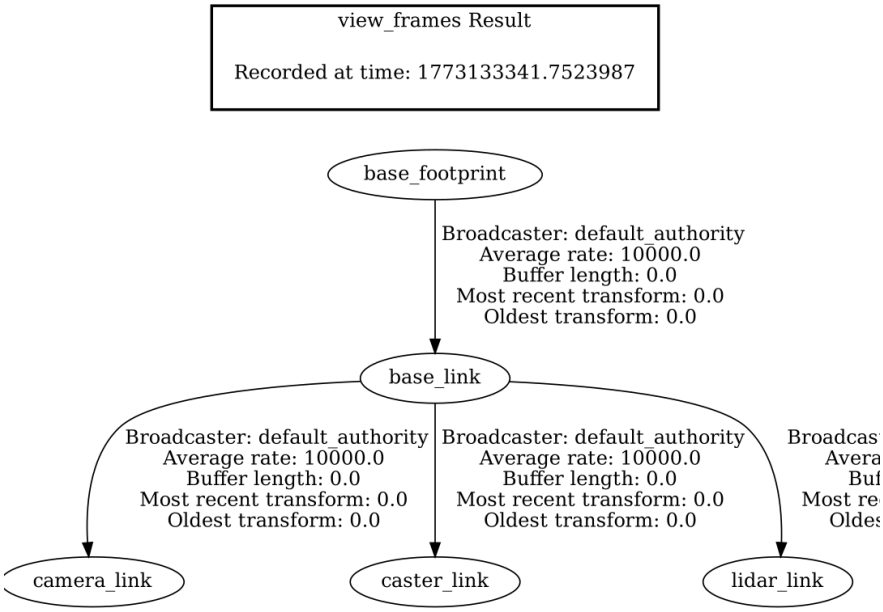
Dentro de la carpeta del paquete se crean subcarpetas que serán necesarias para incroporar los ficheros que describan al robot y a los sensores que se incroporen más adelante. Serán necesarias las carpetas de ‘launch’ y de ‘urdf’.

En la carpeta ‘urdf’ se crean primero el robot en formato ‘robot.urdf’, se describe la geometría del cuerpo y de las ruedas, además de las posiciones relativas y la jerarquía.

En la imágen se puede ver que cuenta con 2 ruedas motrices, una a cada lado y una rueda caster (loca) la cual es únicamente para proporcionar estabilidad.



En la siguiente se puede ver el árbol de jerarquía mencionado. Domina un base_footprint (para centrar el robot en rviz) seguido del base_link (ubicado en el bloque central) y a la misma altura las 3 ruedas.



Archivo robot.urdf

Modificación del archivo a xacro

Con el fin de modularizar se crea un archivo con extensión '.xacro' a partir del código original del robot.urdf. Se define como funciones tipo xacro la geometría original y se deja como variables de entrada las posiciones y los nombres que van a recibir.

```
<xacro:rueda_macro prefix="derecha" y_pos="-${offset_ruedas_y}" />
<xacro:rueda_macro prefix="izquierda" y_pos="${offset_ruedas_y}" />
<xacro:caster_macro prefix="caster" x="-0.2" y="0" z="-0.15" />
```

robot.urdf.xacro

Ahora además se precisan los ficheros de descripción del robot para poder emplear el simulador Gazebo, los cuales son 'materials.xacro' y 'inercias.xacro'.

En el de materiales se incluyen las descripciones de los colores.

El archivo de inercias contempla las descripciones de características físicas que presentan cada una de las piezas que componen al robot (ruedas y bloque central).

En ambos casos se definen como xacros.

Por último, se añaden a cada xacro de las piezas las colisiones, los materiales y las inercias.

Paquete bringup

La ejecución del entorno tiene que realizarse mediante un archivo launch que gestione todos los aspectos necesarios para no tener que realizarlos manualmente mediante varios comandos. Para ello, se crea un segundo paquete 'robot_bringup' el cual cuente con los ficheros :

- **Bridge.yaml:**
  Necesario para poder enlazar los topics y conexiones necesarias entre el simulador Gazebo y ROS2. Además hace porsible ver la información de sensores en Rviz.
- **World.sdf**:
  Archivo de configuración del mundo simulación. Para crearlo basta con crear un mundo vacio en gazebo, añadir las geometrías o modelos que se requieran y se exporta en formato .sdf .
- **Config.rviz:**
  Cada vez que se abra Rviz se ejecutará con la configuración específica la cual haya sido guardada/configurada previamente.
- **Robot_bringup.launch.py:**
  Archivo necesario para la ejecución de los comandos necesarios, incluye los directorios, rutas, nodos de ROS2, bridge y configuración de rviz.

Para poder gestoinar todos los nuevos ficheros que se crean deben estar contemplados en el archivo de configuración del paquete de ROS ‘setup.py’.

```
data_files=[
    ('share/ament_index/resource_index/packages',
        ['resource/' + package_name]),
    ('share/' + package_name, ['package.xml']),
    (os.path.join('share', package_name, 'world'), glob('world/*')),
    (os.path.join('share', package_name, 'launch'), glob('launch/*')),
    (os.path.join('share', package_name, 'config'), glob('config/*')),
    (os.path.join('share', package_name, 'rviz'), glob('rviz/*')),
],
```

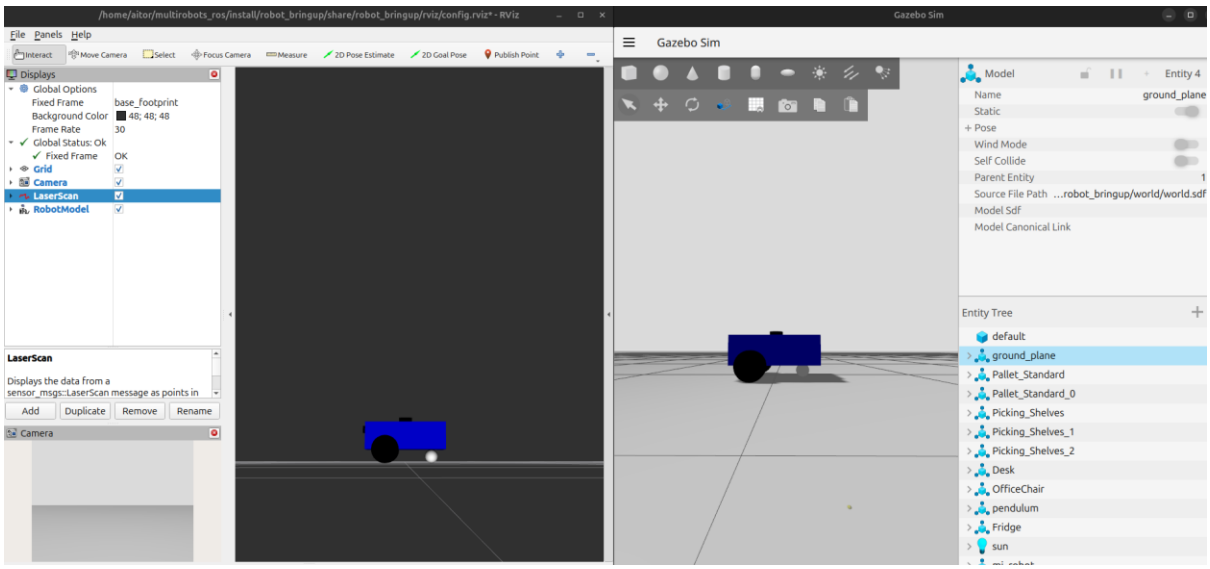
También es necesaria la creación de un archivo **plugins.xacro** en el paquete inicial para poder conectar la simulación con el motor de físicas y la comunicación con ROS (mediante el bridge).

Con todo listo se puede probar la simulación con el siguiente comando:

$ros2 launch robot_bringup robot_bringup.launch.py

(antes es necesario realizar ‘$colcon build’)

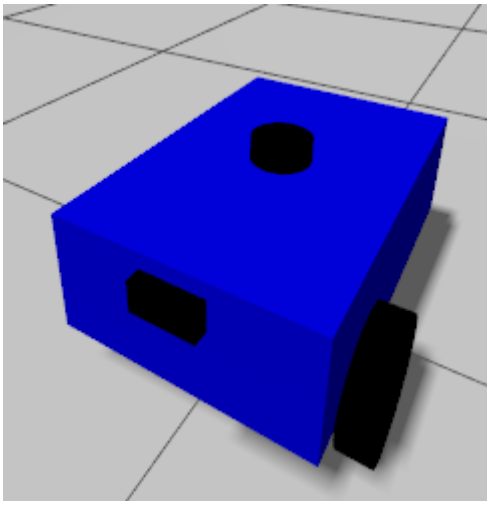
El resultado es el siguiente:



Se puede apreciar que se ve la misma configuración de robot tanto en el visor como en Gazebo.

Incorporación de sensores

Una vez creada toda la estructura del robot y pudiendo ejecutar la simulación y tras comprobar que se puede ver tanto en gazebo como en Rviz toca incorporar los sensores tipo lidar y las cámaras. En este caso el Lidar va encima y la cámara en la parte delantera.



Para lograr el resultado, se crean ambos ficheros dentro de la carpeta donde están los demás archivos de configuración del robot; tienen que ser 'camara.xacro' y 'lidar.xacro'.

Contienen la información de su geometría, su posición en la jerarquía y la posición en el modelo.

Una vez creados se deben incluir en el archivo de descripción del robot:

```
<xacro:include filename="lidar.xacro" />
<xacro:include filename="camara.xacro" />
```

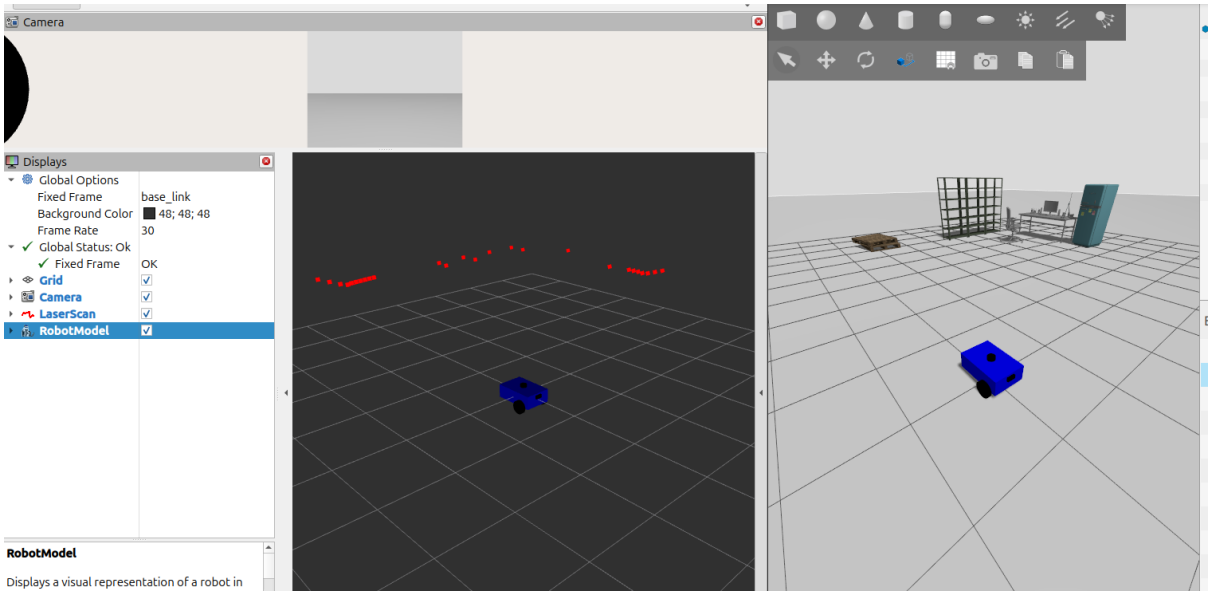
Además se modifica el archivo bridge.yaml para contener la nueva información de los topics de cámara y de lidar. En el caso de la cámara hay que tener en cuenta que se debe incluir el topic /image_raw y /camera_info. Este último para que rviz sea capaz de mostrar la imagen.

También se debe incluir en el archivo del mundo el plugin directamente de los sensores junto a los demás plugins al principio, sino no se simularían:

<plugin name='gz::sim::systems::Sensors' filename='gz-sim-sensors-system'>

<render_engine>ogre2</render_engine></plugin>

Con todo esto ya se tiene el robot en funcionamiento en simulación con el sensor lidar y la cámara. Además de lo que se tenía antes con la odometría ahora y el modelo ahora se puede ver todo el conjunto en rviz funcionando.

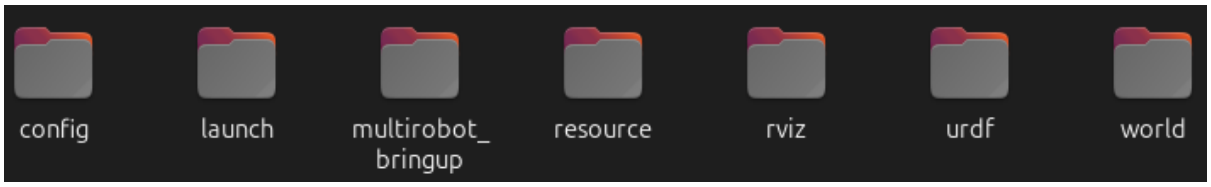


Se aprecia que Rviz muestra la información del sensor láser y la cámara en la parte superior.

Sistema multirobot

Una vez se tiene un único robot funcionando con todo se puede pasar a crear múltiples robots. Para ello, es aconsejable la creación de un nuevo paquete para no sobreescribir sobre el original.

Tras crear el paquete hay que copiar los archivos que teníamos antes divididos en dos paquetes, pero ahora en uno solo dentro de las carpetas correspondientes. También hay que incluir todo en el archivo 'setup.py' como ya se había hecho anteriormente.



Quedan las siguientes carpetas, en este caso interesan las siguientes: config, launch, rviz, urdf y world.

Ahora viene el paso clave, hay que añadir un prefijo a cada pieza que incorpore el robot, en los links, en los joint y en la referencia de gazebo.

Ejemplo para la cámara:

```
<link name="$(arg prefix)camera_link">
<joint name="$(arg prefix)camera_joint" type="fixed">
```

```
<gazebo reference="$(arg prefix)camera_link">
```

(líneas de inicio de la descripción ‘link’, ‘joint’ y ‘gazebo’)

Este prefijo debe añadirse en los siguientes archivos:

- Robot.urdf.xacro
- Camara.xacro
- Lidar.xacro
- Plugins.xacro

El archivo bridge en este caso se tiene que crear ‘al vuelo’ en cada ejecución ya que debe ser dinámico para que contemple el número de robots requerido cada vez.

Por eso, en el archivo ‘launch’ se tiene que crear las ‘tuberías’ necesarias para crear los enlazamientos correspondientes.

El archivo multirobot_bringup.launch.py contempla la creación de los múltiples robots de forma dinámica mediante un bucle el cual crea un robot y asigna una etiqueta a cada uno de los topics que tiene. Se pone el prefijo de ‘robotX’ y sirve de prefijo para estos topics:

```
bridge_config.extend([
    {'ros_topic_name': f'/{robot_name}/cmd_vel', 'gz_topic_name': f'/{robot
    {'ros_topic_name': f'/{robot_name}/odom', 'gz_topic_name': f'/{robot_na
    {'ros_topic_name': f'/{robot_name}/scan', 'gz_topic_name': f'/{robot_na
    {'ros_topic_name': f'/{robot_name}/camera/image_raw', 'gz_topic_name':
    {'ros_topic_name': f'/{robot_name}/camera/camera_info', 'gz_topic_name'
    {'ros_topic_name': f'/{robot_name}/joint_states', 'gz_topic_name': f'/{
])
```

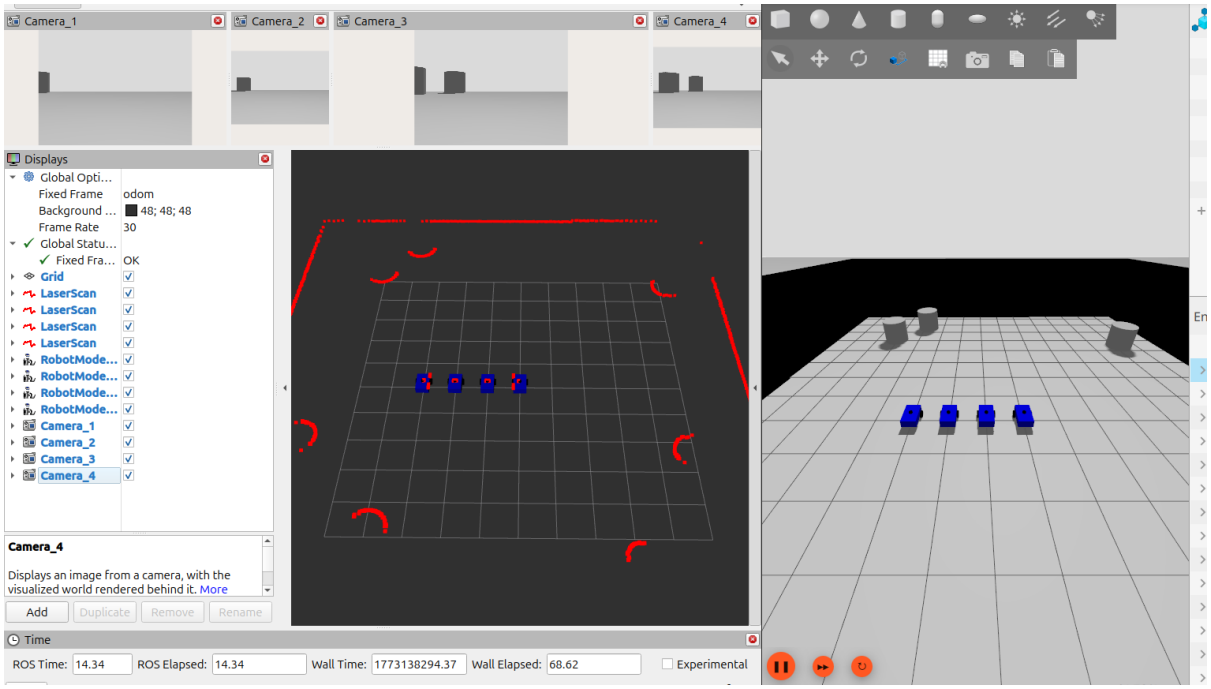
Otro factor importante es que cada uno de ellos haga ‘spawn’ separado para el visor y gazebo modificando esta posición. En este caso la posición y:

```
arguments=[
'--x', '0.0',
'--y', str(y_pose),
'--z', '0.0',
```

Luego con toda la información se crea el archivo temporal de multirobot_bridge.yaml.

Resumiendo, para que se pueda generar el sistema multirobot, se tiene que poner a cada uno de los robots que se quiera crear un prefijo para poder diferenciar de donde proviene cada uno de los topcis o hacia cuál de ellos se dirige. Mediante el bucle for se define el número de repeticiones viene dado por el archivo que se pasa por parámetro al ejecutar el comando launch; luego se crea el bridge ‘al vuelo’ y se lanza la simulación y el visor Rviz. El archivo launch también se conecta con la descripción del

robot la cual hemos modificado para que acepte múltiples robots mediante los prefijos.



El árbol de jerarquías para 2 robots quedaría de la siguiente forma:

```
map (Fixed Frame Global)
  |
  ├── robot1/odom (Odometría del Robot 1)
  |     |
  |     └── robot1/base_footprint (Proyección en el suelo)
  |           |
  |           └── robot1/base_link (Chasis del robot)
  |                 |
  |                 ├── robot1/camera_link
  |                 ├── robot1/lidar_link
  |                 ├── robot1/rueda_derecha_link
  |                 ├── robot1/rueda_izquierda_link
  |                 └── robot1/caster_link
  |
  └── robot2/odom (Odometría del Robot 2)
        |
        └── robot2/base_footprint (Proyección en el suelo)
              |
              └── robot2/base_link (Chasis del robot)
                    |
                    ├── robot2/camera_link
                    ├── robot2/lidar_link
                    ├── robot2/rueda_derecha_link
                    ├── robot2/rueda_izquierda_link
                    └── robot2/caster_link
```